



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 03

Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The state space contains all valid grid positions. The search tree shows the different paths explored from the starting position.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

The reached set prevents revisiting states and creating cycles. Without it, DFS could loop between the same cells and never find the food.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS checks all nearby positions before exploring farther positions. Therefore, it finds the shortest path when every movement has the same cost.

DFS follows one path as deeply as possible before trying another. It may choose a long, winding route even when a shorter route exists. Therefore, DFS is not guaranteed to be optimal.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

BFS uses $O(b^d)$ memory because it stores many frontier nodes. DFS uses $O(bm)$ memory and stores fewer nodes. Therefore, BFS and UCS may run out of memory on a large grid, while DFS uses less memory but may find a longer path.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation



FACULTY OF COMPUTING